

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN I



BÁO CÁO BÀI TẬP LỚN
CƠ SỞ DỮ LIỆU ĐA PHƯƠNG TIỆN
ĐỀ TÀI XÂY DỰNG HỆ CƠ SỞ DỮ LIỆU LƯU TRỮ VÀ TÌM
KIẾM TIẾNG ĐỘNG VẬT

Giảng viên: TS. Nguyễn Đình Hóa

Nhóm học phần: 02

Danh sách thành viên - Nhóm: 25

Họ và tên	Mã sinh viên
Phạm Văn Sơn	B22DCCN697
Nguyễn Tiến Hưng	B22DCCN415
Nguyễn Thị Huế	B22DCCN355

Hà Nội, 05/2026

LỜI CẢM ƠN

Trước tiên, chúng em xin gửi lời cảm ơn chân thành đến thầy Nguyễn Đình Hóa - giảng viên bộ môn, đã dành thời gian xem xét và đánh giá bài báo cáo của nhóm. Sự hướng dẫn và những góp ý quý báu của thầy đã giúp chúng em hoàn thiện bài báo cáo này một cách tốt nhất.

Chúng em cũng xin gửi lời cảm ơn đến các thành viên trong nhóm, những người đã cùng nhau hợp tác, đóng góp ý tưởng và nỗ lực hết mình để hoàn thành bài tập. Mỗi người trong nhóm đều có những đóng góp quan trọng, và sự đoàn kết, phối hợp chặt chẽ giữa các thành viên là yếu tố quyết định giúp chúng em hoàn thành công việc này.

Cảm ơn các bạn trong nhóm đã luôn hỗ trợ, chia sẻ và cùng nhau vượt qua mọi thử thách trong suốt quá trình làm việc. Chúng em hy vọng rằng bài báo cáo này sẽ đạt được kết quả tốt, phản ánh công sức mà nhóm đã bỏ ra.

Xin chân thành cảm ơn!

MỤC LỤC

DANH MỤC HÌNH ẢNH.....	1
CHƯƠNG 1. GIỚI THIỆU	1
1.1. Đặt vấn đề	1
1.2. Mục tiêu đề tài.....	1
1.3. Phạm vi	2
CHƯƠNG 2. CƠ SỞ LÝ THUYẾT VÀ PHƯƠNG PHÁP	4
2.1. Tổng quan CSDL đa phương tiện	4
2.2. Bài toán tìm kiếm âm thanh theo nội dung	4
2.3. Lựa chọn đặc trưng âm thanh	5
2.3.1. <i>Zero Crossing Rate (ZCR)</i>	6
2.3.2. <i>Root Mean Square Energy (RMSE)</i>	7
2.3.3. <i>MFCC (Mel-Frequency Cepstral Coefficients)</i>	7
2.3.4. <i>Spectral Centroid</i>	8
2.3.5. <i>Spectral Bandwidth</i>	8
2.3.6. <i>Spectral Roll-off</i>	9
CHƯƠNG 3. XÂY DỰNG BỘ DỮ LIỆU VÀ TRÍCH RÚT ĐẶC TRƯNG	11
3.1. Bộ dữ liệu	11
3.2. Tiền xử lý	13
3.2.1. <i>Chuẩn hóa dữ liệu</i>	14
3.2.2. <i>Xử lý tín hiệu âm thanh</i>	15
3.3. Trích rút đặc trưng.....	20
3.3.1. <i>Trích rút đặc trưng thủ công bằng NumPy</i>	20
3.3.2. <i>Trích rút đặc trưng bằng thư viện Librosa</i>	21
3.3.3. <i>Tạo vector đặc trưng</i>	22
CHƯƠNG 4. THIẾT KẾ CƠ SỞ DỮ LIỆU VÀ HỆ THỐNG TÌM KIẾM.....	23
4.1. Thiết kế CSDL đặc trưng	23
4.2. Sơ đồ khối hệ thống	24

4.3. Chuẩn hóa đặc trưng.....	25
4.4. Phương pháp đo độ tương đồng.....	27
4.5. Quy trình tìm kiếm	28
4.6. Kết quả trung gian.....	29
CHƯƠNG 5. DEMO VÀ ĐÁNH GIÁ.....	30
5.5. Demo hệ thống.....	30
5.6. Đánh giá	32
KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN.....	34

DANH MỤC HÌNH ẢNH

Hình 4.1. CSDL đặc trưng	23
Hình 4.2. Sơ đồ khối hệ thống	24
Hình 4.3. CSDL đặc trưng sau chuẩn hóa	26
Hình 4.4. Quy trình tìm kiếm.....	28
Hình 5.1. Giao diện chính hiển thị hệ thống tìm kiếm tiếng động vật.....	30
Hình 5.2. Giao diện hệ thống khi chọn file tìm kiếm từ thiết bị cục bộ	30
Hình 5.3. Giao diện hệ thống khi ấn nút “Tìm kiếm”.....	31
Hình 5.4. Log hệ thống khi chạy ứng dụng web (app.py) trên localhost	32

CHƯƠNG 1. GIỚI THIỆU

1.1. Đặt vấn đề

Trong bối cảnh chuyển đổi số và sự phát triển mạnh mẽ của các hệ thống dữ liệu đa phương tiện, việc khai thác và xử lý dữ liệu âm thanh ngày càng đóng vai trò quan trọng trong nhiều lĩnh vực như sinh học, bảo tồn đa dạng sinh học, nông nghiệp thông minh và giám sát môi trường. Trong đó, âm thanh của động vật là một nguồn dữ liệu giàu thông tin, có khả năng phản ánh đặc điểm sinh học, hành vi và điều kiện môi trường sống của từng loài.

Tuy nhiên, các hệ thống lưu trữ và quản lý âm thanh hiện nay chủ yếu dựa trên metadata như tên file, nhãn loài hoặc mô tả thủ công. Cách tiếp cận này tồn tại nhiều hạn chế, bao gồm tính phụ thuộc vào con người, thiếu tính nhất quán và đặc biệt không cho phép tìm kiếm theo nội dung âm thanh thực tế. Trong nhiều trường hợp, người dùng chỉ có một đoạn âm thanh đầu vào nhưng không biết thông tin về loài, hoặc có nhu cầu tìm kiếm các âm thanh tương tự để phục vụ nghiên cứu, thì các hệ thống truyền thống không đáp ứng được yêu cầu.

Bên cạnh đó, đặc trưng âm thanh của động vật mang tính phức tạp và biến thiên theo nhiều yếu tố như môi trường ghi âm, trạng thái hành vi hoặc đặc điểm sinh học của từng cá thể. Do đó, việc biểu diễn âm thanh dưới dạng các đặc trưng số có khả năng phản ánh bản chất tín hiệu là yếu tố then chốt để phục vụ cho các bài toán so sánh và tìm kiếm.

Sự phát triển của các kỹ thuật xử lý tín hiệu số, trích rút đặc trưng âm thanh (như MFCC, spectral features) và các hệ cơ sở dữ liệu đa phương tiện đã tạo điều kiện thuận lợi để xây dựng các hệ thống tìm kiếm dựa trên nội dung (Content-Based Audio Retrieval). Xuất phát từ những vấn đề trên, đề tài tập trung xây dựng một hệ cơ sở dữ liệu lưu trữ và tìm kiếm tiếng động vật dựa trên nội dung âm thanh, nhằm nâng cao hiệu quả khai thác và sử dụng dữ liệu âm thanh trong thực tiễn.

1.2. Mục tiêu đề tài

Mục tiêu tổng quát của đề tài là xây dựng một hệ thống cơ sở dữ liệu đa phương tiện có khả năng lưu trữ, trích rút đặc trưng và tìm kiếm tiếng động vật dựa trên nội dung âm thanh.

Cụ thể, đề tài hướng tới các mục tiêu sau:

Thứ nhất, xây dựng bộ dữ liệu gồm ít nhất 500 file âm thanh tiếng động vật, đảm bảo mỗi file chứa âm thanh của một loài và có chất lượng phù hợp để trích rút đặc trưng.

Thứ hai, lựa chọn và xây dựng tập thuộc tính (đặc trưng âm thanh) có khả năng biểu diễn và phân biệt các loại tiếng động vật, đồng thời phân tích giá trị thông tin của từng đặc trưng.

Thứ ba, thiết kế và xây dựng hệ thống trích rút đặc trưng từ dữ liệu âm thanh và lưu trữ các đặc trưng này dưới dạng cơ sở dữ liệu có cấu trúc.

Thứ tư, xây dựng cơ chế tìm kiếm dựa trên độ tương đồng giữa các đặc trưng, cho phép truy vấn với đầu vào là một file âm thanh và trả về các file tương tự nhất.

Thứ năm, đánh giá hiệu quả của hệ thống thông qua kết quả tìm kiếm và khả năng phản ánh đúng sự tương đồng về nội dung âm thanh.

1.3. Phạm vi

Đề tài tập trung vào việc xây dựng hệ thống cơ sở dữ liệu và tìm kiếm tiếng động vật dựa trên nội dung âm thanh, với phạm vi cụ thể như sau:

Về dữ liệu: Sử dụng tập dữ liệu gồm tối thiểu 500 file âm thanh tiếng động vật, được thu thập từ các nguồn công khai hoặc tự xây dựng. Mỗi file chỉ chứa âm thanh của một loài và có độ dài đủ để trích rút đặc trưng.

Về đặc trưng: Tập trung vào các đặc trưng âm thanh phổ biến và có hiệu quả trong nhận diện như MFCC, zero-crossing rate, spectral centroid, spectral roll-off,... Không đi sâu vào các mô hình học sâu phức tạp.

Về hệ thống: Xây dựng hệ thống tìm kiếm dựa trên so sánh độ tương đồng giữa các vector đặc trưng (ví dụ: khoảng cách Euclidean, cosine similarity). Hệ thống hoạt động ở mức mô phỏng/ứng dụng cơ bản, chưa tối ưu cho quy mô lớn.

Về chức năng: Hệ thống cho phép nhập một file âm thanh đầu vào và trả về danh sách 5 file âm thanh tương tự nhất, bao gồm cả trường hợp âm thanh truy vấn có hoặc không có trong cơ sở dữ liệu.

Về giới hạn: Đề tài không tập trung vào xây dựng hệ thống nhận dạng chính xác tên loài (classification hoàn chỉnh), mà chủ yếu hướng tới bài toán tìm kiếm theo nội

dung (retrieval). Ngoài ra, các yếu tố như nhiễu môi trường hoặc chất lượng ghi âm chưa được xử lý ở mức nâng cao.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT VÀ PHƯƠNG PHÁP

2.1. Tổng quan CSDL đa phương tiện

Cơ sở dữ liệu đa phương tiện là sự mở rộng của các hệ cơ sở dữ liệu truyền thống, nhằm đáp ứng nhu cầu lưu trữ và khai thác các loại dữ liệu có tính chất phức tạp như hình ảnh, âm thanh, video và văn bản. Điểm khác biệt lớn nằm ở chỗ dữ liệu không còn đơn thuần là số hay ký tự, mà là sự kết hợp của nhiều loại phương tiện khác nhau, đòi hỏi cách tổ chức và xử lý linh hoạt hơn.

Xét về đặc điểm, dữ liệu đa phương tiện thường được chia thành hai nhóm chính là phương tiện tĩnh và phương tiện động. Trong khi phương tiện tĩnh như hình ảnh hay văn bản không phụ thuộc vào thời gian, thì phương tiện động như âm thanh và video lại gắn liền với yếu tố thời gian và cần được xử lý theo dòng liên tục. Điều này khiến việc lưu trữ và quản lý trở nên phức tạp hơn so với dữ liệu truyền thống.

Trong các hệ cơ sở dữ liệu thông thường, dữ liệu được tổ chức theo bảng với các kiểu dữ liệu rõ ràng. Tuy nhiên, cách tiếp cận này không hoàn toàn phù hợp với dữ liệu đa phương tiện. Trên thực tế, các file âm thanh hoặc video thường được lưu dưới dạng các khối dữ liệu lớn, và hệ thống chỉ quản lý ở mức lưu trữ mà không hiểu được nội dung bên trong.

Để khắc phục hạn chế này, các hệ thống đa phương tiện thường bổ sung thêm bước trích rút đặc trưng. Thay vì xử lý trực tiếp dữ liệu thô, hệ thống sẽ chuyển dữ liệu thành các vector đặc trưng, từ đó có thể thực hiện so sánh và tìm kiếm hiệu quả hơn. Việc so sánh trong trường hợp này không còn mang tính chính xác tuyệt đối, mà dựa trên mức độ tương tự giữa các đối tượng.

Ngoài ra, do dữ liệu đa phương tiện thường có dạng nhiều chiều, việc tìm kiếm hiệu quả đòi hỏi sử dụng các cấu trúc chỉ số phù hợp. Một số cấu trúc phổ biến có thể kể đến như cây k-d, cây tứ phân hoặc các dạng chỉ số không gian khác, giúp giảm thời gian truy vấn và cải thiện hiệu năng hệ thống.

2.2. Bài toán tìm kiếm âm thanh theo nội dung

Bài toán tìm kiếm âm thanh theo nội dung xuất phát từ những hạn chế của các hệ thống tìm kiếm truyền thống. Trong các hệ thống này, việc truy vấn chủ yếu dựa vào metadata hoặc mô tả văn bản. Tuy nhiên, cách tiếp cận này chưa thực sự hiệu quả đối

với dữ liệu đa phương tiện, vì nội dung thực tế của âm thanh không thể được thể hiện đầy đủ chỉ thông qua mô tả bằng chữ.

Từ đó, phương pháp tìm kiếm theo nội dung (Content-Based Retrieval) được đề xuất, trong đó việc tìm kiếm không còn dựa vào thông tin bên ngoài mà dựa trực tiếp vào các đặc trưng của dữ liệu. Đối với âm thanh, các đặc trưng này có thể là phổ tần số, năng lượng tín hiệu hoặc các hệ số biểu diễn tín hiệu, giúp phản ánh bản chất âm học của dữ liệu.

Một điểm khác biệt quan trọng so với tìm kiếm truyền thống là việc so sánh không còn mang tính chính xác tuyệt đối mà chuyển sang đánh giá mức độ tương tự. Điều này có nghĩa là hệ thống không tìm kiếm các kết quả “giống hệt”, mà ưu tiên các đối tượng có mức độ giống nhau cao nhất so với dữ liệu đầu vào. Vì vậy, cần có các phương pháp đo lường độ tương đồng giữa các vector đặc trưng để thực hiện quá trình này.

Về mặt tổng thể, hệ thống tìm kiếm theo nội dung thường gồm hai giai đoạn chính. Trước hết, dữ liệu trong cơ sở dữ liệu sẽ được xử lý để trích rút đặc trưng và lưu trữ dưới dạng đã được tổ chức. Sau đó, khi có truy vấn đầu vào, hệ thống cũng tiến hành trích rút đặc trưng tương tự và thực hiện so sánh với dữ liệu đã có để tìm ra các kết quả phù hợp nhất.

Trong phạm vi đề tài, đầu vào của hệ thống là một file âm thanh tiếng động vật, và đầu ra là danh sách các file có nội dung tương tự nhất. Để thực hiện được điều này, hệ thống cần giải quyết ba vấn đề chính, bao gồm biểu diễn âm thanh dưới dạng đặc trưng số, xây dựng cơ chế đo độ tương đồng và tổ chức dữ liệu sao cho việc tìm kiếm đạt hiệu quả cao.

Tuy nhiên, một thách thức lớn của bài toán này là khái niệm “tương tự” trong âm thanh mang tính tương đối và phụ thuộc nhiều vào ngữ cảnh. Hai âm thanh có thể gần giống nhau về mặt tín hiệu nhưng lại khác nhau về ý nghĩa, hoặc ngược lại. Vì vậy, việc lựa chọn đặc trưng phù hợp đóng vai trò rất quan trọng, ảnh hưởng trực tiếp đến chất lượng kết quả tìm kiếm của hệ thống.

2.3. Lựa chọn đặc trưng âm thanh

Trong bài toán tìm kiếm âm thanh theo nội dung, mục tiêu chính không chỉ là lưu trữ dữ liệu mà còn phải biểu diễn được âm thanh dưới dạng có thể so sánh được. Vì vậy,

việc lựa chọn đặc trưng đóng vai trò quan trọng, quyết định trực tiếp đến khả năng đánh giá độ tương đồng giữa các tín hiệu.

Các đặc trưng được nhóm lựa chọn trong đề tài này bao gồm cả miền thời gian và miền tần số, nhằm đảm bảo phản ánh được nhiều khía cạnh khác nhau của âm thanh. Thay vì phụ thuộc vào một đặc trưng duy nhất, việc kết hợp nhiều đặc trưng giúp hệ thống biểu diễn tín hiệu một cách đầy đủ và chính xác hơn.

2.3.1. Zero Crossing Rate (ZCR)

Zero Crossing Rate là đặc trưng phản ánh số lần tín hiệu âm thanh đổi dấu trong một khoảng thời gian, qua đó thể hiện mức độ dao động của tín hiệu theo miền thời gian. Nói cách khác, đặc trưng này cho biết tín hiệu thay đổi nhanh hay chậm, từ đó gián tiếp phản ánh đặc điểm tần số của âm thanh.

Công thức tính ZCR được biểu diễn như sau:

$$ZCR = \frac{1}{2N} \sum_{n=1}^N |sign(x[n]) - sign(x[n-1])| \quad (1)$$

Xét về bản chất, những tín hiệu chứa nhiều thành phần tần số cao thường có dạng sóng dao động liên tục, dẫn đến số lần cắt trục 0 lớn và giá trị ZCR cao. Ngược lại, các tín hiệu trầm, ổn định sẽ có ít lần đổi dấu hơn, tương ứng với giá trị ZCR thấp. Vì vậy, ZCR có thể được xem là một chỉ báo ban đầu về tính chất “sắc” hay “trầm” của âm thanh.

Trong bài toán tìm kiếm âm thanh theo nội dung, việc so sánh cần dựa trên các đặc điểm có thể lượng hóa được. Tuy nhiên, nếu chỉ sử dụng các đặc trưng phổ, hệ thống có thể bỏ qua những khác biệt cơ bản về dạng sóng. Do đó, ZCR được nhóm lựa chọn nhằm bổ sung thông tin về mức độ dao động của tín hiệu.

Nhờ đặc trưng này, hệ thống có thể phân biệt nhanh các âm thanh có đặc điểm khác nhau ở mức tổng quát, ví dụ như âm thanh có nhiều thành phần cao tần so với âm thanh trầm. Từ đó, việc đánh giá độ tương đồng trở nên chính xác hơn khi kết hợp với các đặc trưng khác.

2.3.2. Root Mean Square Energy (RMSE)

RMSE là đặc trưng thể hiện năng lượng trung bình của tín hiệu âm thanh trong một khoảng thời gian, qua đó phản ánh cường độ của âm thanh.

Công thức tính RMSE:

$$RMSE = \sqrt{\frac{1}{N} \sum_{n=1}^N x[n]^2} \quad (2)$$

Xét về tín hiệu, năng lượng phụ thuộc trực tiếp vào biên độ sóng. Khi biên độ lớn, năng lượng tăng lên và RMSE có giá trị cao; ngược lại, khi tín hiệu yếu, RMSE sẽ thấp. Điều này giúp đặc trưng này biểu diễn rõ ràng mức độ “mạnh – yếu” của âm thanh.

Áp dụng với trường hợp này, nếu chỉ xét đến hình dạng phổ mà bỏ qua năng lượng, hệ thống có thể đánh giá chưa chính xác độ tương đồng. Hai âm thanh có cấu trúc phổ giống nhau nhưng chênh lệch lớn về cường độ vẫn có thể được xem là khác nhau về mặt thực tế.

Vì vậy, RMSE được nhóm lựa chọn nhằm bổ sung thông tin về năng lượng, giúp việc so sánh giữa các tín hiệu trở nên đầy đủ hơn. Khi hai âm thanh có giá trị RMSE gần nhau, có thể suy ra rằng chúng có mức năng lượng tương tự, từ đó tăng độ tin cậy khi đánh giá độ giống nhau.

2.3.3. MFCC (Mel-Frequency Cepstral Coefficients)

MFCC là đặc trưng biểu diễn cấu trúc phổ của tín hiệu dựa trên thang tần số Mel, phù hợp với cách con người cảm nhận âm thanh.

Khác với các đặc trưng đơn lẻ, MFCC không chỉ phản ánh một yếu tố cụ thể mà tổng hợp toàn bộ thông tin phổ của tín hiệu thành một tập hệ số. Điều này cho phép MFCC nắm bắt được đặc điểm âm sắc – yếu tố quan trọng giúp phân biệt các loại âm thanh khác nhau.

Công thức tổng quát của MFCC:

$$MFCC(k) = \sum_{n=1}^N \log(S(n)) \cdot \cos\left(\frac{\pi k(n - 0.5)}{N}\right) \quad (3)$$

Xét về bản chất, hai âm thanh được coi là giống nhau chủ yếu dựa trên sự tương đồng về cấu trúc phổ, chứ không chỉ dựa vào năng lượng hay tần số riêng lẻ. MFCC cho phép biểu diễn cấu trúc này một cách cô đọng nhưng vẫn giữ được thông tin quan trọng.

MFCC được nhóm lựa chọn làm đặc trưng cốt lõi trong đề tài này, vì nó có khả năng phản ánh trực tiếp nội dung của âm thanh. Khi hai tín hiệu có vector MFCC gần nhau, có thể suy ra rằng chúng có cấu trúc phổ tương tự.

Vì vậy, MFCC đóng vai trò chính trong việc xác định độ tương đồng giữa các âm thanh, trong khi các đặc trưng khác đóng vai trò hỗ trợ nhằm hoàn thiện việc biểu diễn tín hiệu.

2.3.4. Spectral Centroid

Spectral Centroid là đặc trưng biểu diễn vị trí trung bình của năng lượng trong phổ tần số, phản ánh xu hướng âm thanh nghiêng về tần số cao hay thấp.

Công thức:

$$Centroid = \frac{\sum f(k) \cdot X(k)}{\sum X(k)} \quad (4)$$

Khi năng lượng tập trung ở vùng tần số cao, centroid sẽ có giá trị lớn, thể hiện âm thanh sắc và sáng. Ngược lại, khi năng lượng tập trung ở tần số thấp, centroid sẽ nhỏ, thể hiện âm thanh trầm hơn.

Việc chỉ sử dụng MFCC khi phân tích có thể chưa đủ để thể hiện rõ sự khác biệt về xu hướng tần số tổng thể. Do đó, Spectral Centroid được nhóm lựa chọn nhằm bổ sung thông tin về vị trí phân bố năng lượng trong phổ.

Hai tín hiệu có centroid gần nhau thường có đặc điểm tần số tương tự ở mức tổng thể. Vì vậy, đặc trưng này giúp hệ thống phân biệt các âm thanh có xu hướng “cao – thấp” khác nhau, từ đó hỗ trợ quá trình đánh giá độ tương đồng.

2.3.5. Spectral Bandwidth

Spectral Bandwidth là đặc trưng phản ánh mức độ phân tán năng lượng của tín hiệu trong miền tần số, qua đó cho biết độ rộng của phổ âm thanh. Cụ thể, đặc trưng này đo lường mức độ mà năng lượng của tín hiệu được phân bố xung quanh tần số trung tâm (spectral centroid), từ đó thể hiện phạm vi tần số mà tín hiệu chiếm giữ.

Công thức:

$$Bandwidth = \sqrt{\frac{\sum (f(k) - Centroid)^2 \cdot X(k)}{\sum X(k)}} \quad (5)$$

Khi năng lượng tập trung trong một dải tần hẹp, giá trị bandwidth sẽ thấp, thể hiện âm thanh có cấu trúc đơn giản và ổn định. Ngược lại, khi năng lượng trải rộng trên nhiều thành phần tần số, bandwidth sẽ cao, phản ánh âm thanh có cấu trúc phức tạp hơn hoặc chứa nhiều thành phần dao động khác nhau.

Việc so sánh âm thanh không chỉ dựa vào vị trí tần số trung tâm mà còn cần xét đến cách năng lượng được phân bố trong phổ. Hai tín hiệu có thể có cùng spectral centroid nhưng vẫn khác nhau đáng kể về mức độ phân tán năng lượng. Do đó, nếu chỉ sử dụng centroid, hệ thống có thể đánh giá chưa chính xác mức độ tương đồng.

Vì vậy, Spectral Bandwidth được nhóm lựa chọn nhằm bổ sung thông tin về độ rộng của phổ, giúp mô tả chi tiết hơn cấu trúc tần số của tín hiệu. Thông qua đặc trưng này, hệ thống có thể phân biệt các âm thanh có cùng xu hướng tần số nhưng khác nhau về độ phức tạp. Từ đó, việc tính toán độ tương đồng giữa các vector đặc trưng trở nên đầy đủ và chính xác hơn.

2.3.6. Spectral Roll-off

Spectral Roll-off là đặc trưng xác định tần số mà tại đó phần lớn năng lượng của tín hiệu đã được tích lũy (thường là khoảng 85%).

Công thức:

$$\sum_{k=1}^{k_r} X(k) = 0.85 \sum_{k=1}^K X(k) \quad (6)$$

Roll-off phản ánh giới hạn phía trên của phổ tần số, cho biết năng lượng của tín hiệu phân bố đến mức nào trong miền tần số. Nếu roll-off cao, tín hiệu chứa nhiều thành phần tần số cao; ngược lại, roll-off thấp cho thấy năng lượng chủ yếu tập trung ở vùng tần số thấp.

Nếu chỉ xét mỗi centroid và bandwidth thì vẫn chưa đủ để mô tả đầy đủ phân bố năng lượng của tín hiệu. Hai âm thanh có thể có cùng centroid và bandwidth nhưng khác nhau ở phần tần số cao.

Vì vậy, Spectral Roll-off được nhóm lựa chọn nhằm bổ sung thông tin về giới hạn phổ, giúp mô tả đầy đủ hơn cách năng lượng được phân bố. Nhờ đó, hệ thống có thể phân biệt các tín hiệu có sự khác biệt ở vùng tần số cao, từ đó nâng cao độ chính xác khi đánh giá độ tương đồng.

CHƯƠNG 3. XÂY DỰNG BỘ DỮ LIỆU VÀ TRÍCH RÚT ĐẶC TRƯNG

3.1. Bộ dữ liệu

Bộ dữ liệu bao gồm 505 file âm thanh về 12 loài động khác nhau. Tất cả các file đều chỉ chứa âm thanh của một loài động và không có âm thanh nền. Mỗi file có độ dài từ 2 đến 10 giây và được lưu dưới dạng wav/mp3/flac.

Dữ liệu âm thanh được nhóm chúng em thu thập hoàn toàn ở trên trang freesound.org, BBC Sound Effects, Zapsplat, soundbible.com, mixkit.co, đây là các nền tảng chia sẻ âm thanh miễn phí trên internet, chúng em đã lọc kỹ các file âm thanh và đảm bảo rằng mỗi file chỉ chứa âm thanh của một tiếng của một loài động vật.

a, Bear (con gấu)

- Gồm 45 file âm thanh.
- Đặc điểm: Tiếng kêu của gấu là tiếng gầm hoặc gừ, có âm lượng lớn và mang tính uy lực. Âm thanh thường trầm, dày và có độ rung mạnh, thể hiện sự đe dọa hoặc cảnh báo. Tiếng kêu của gấu có thể kéo dài hoặc ngắt quãng, với biên độ dao động không đều, đặc biệt tăng mạnh khi con vật ở trạng thái kích động.

b, Cat (con mèo)

- Gồm 45 file âm thanh.
- Đặc điểm: Tiếng kêu của mèo là tiếng “meo”, có âm lượng nhỏ đến trung bình và mang tính mềm mại. Âm thanh khá cao, mượt và có thể kéo dài hoặc ngắt quãng tùy trạng thái. Tiếng mèo thường có sự biến đổi nhẹ về cao độ và biên độ, thể hiện nhiều cảm xúc khác nhau như đòi hỏi, thân thiện hoặc khó chịu.

c, Cow (con bò)

- Gồm 45 file âm thanh.
- Đặc điểm: Tiếng kêu của bò là tiếng “ò”, có âm lượng lớn và kéo dài. Âm thanh trầm, vang và ổn định, thường ít biến đổi về cao độ trong một lần phát âm. Biên độ dao động tương đối đều, tạo cảm giác chậm rãi và rõ ràng.

d, Dog (con chó)

- Gồm 45 file âm thanh.
- Đặc điểm: Tiếng kêu của chó là tiếng sủa, có âm lượng từ trung bình đến lớn và mang tính cảnh báo cao. Âm thanh thường sắc, rõ và có nhịp điệu nhanh, lặp lại liên tục. Tiếng sủa có sự dao động mạnh về biên độ và cao độ tùy theo mức độ kích thích hoặc trạng thái cảm xúc.

e, Goat (con dê)

- Gồm 45 file âm thanh.
- Đặc điểm: Tiếng kêu của dê là tiếng “be be”, có âm lượng trung bình và âm sắc khá cao. Âm thanh thường ngắt quãng, lặp lại theo chuỗi và có sự dao động nhẹ về cao độ. Tiếng dê mang tính đặc trưng, hơi chói nhưng không quá mạnh.

f, Hippo (con hà mã)

- Gồm 25 file âm thanh.
- Đặc điểm: Tiếng kêu của hà mã là tiếng gầm hoặc khịt mạnh, có âm lượng rất lớn và vang xa. Âm thanh trầm, dày và có độ rung mạnh, thường phát ra thành từng nhịp ngắn hoặc chuỗi âm. Biên độ dao động lớn, thể hiện rõ tính lãnh thổ và cảnh báo.

g, Horse (con ngựa)

- Gồm 45 file âm thanh.
- Đặc điểm: Tiếng kêu của ngựa là tiếng hí, có âm lượng lớn và vang. Âm thanh tương đối cao, kéo dài và có sự biến đổi rõ về cao độ trong một lần phát âm. Tiếng hí thường bắt đầu mạnh, sau đó giảm dần, tạo cảm giác lan tỏa.

h, Leopard (con báo)

- Gồm 25 file âm thanh.
- Đặc điểm: Tiếng kêu của báo hoa mai là tiếng gầm hoặc rít, có âm lượng lớn và mang tính đe dọa. Âm thanh trầm xen lẫn âm cao, thường ngắt quãng và có sự thay đổi mạnh về biên độ. Tiếng kêu có thể ngắn nhưng sắc và rõ, thể hiện tính săn mồi.

i, Lion (con sư tử)

- Gồm 45 file âm thanh.

- Đặc điểm: Tiếng kêu của sư tử là tiếng gầm, có âm lượng rất lớn và cực kỳ uy lực. Âm thanh trầm, dày và vang xa, thường phát ra theo từng đợt rõ rệt. Biên độ dao động mạnh, ít thay đổi trong mỗi đợt gầm nhưng có nhịp điệu rõ, thể hiện vị thế thống trị.

k, Monkey (con khỉ)

- Gồm 45 file âm thanh.
- Đặc điểm: Tiếng kêu của khỉ là những âm thanh chít chít hoặc la hét, có âm lượng trung bình đến lớn. Âm thanh thường cao, sắc và thay đổi nhanh về cao độ. Tiếng khỉ có tính lặp lại cao, nhịp nhanh và biên độ dao động lớn, thể hiện sự linh hoạt và giao tiếp trong bầy.

l, Pig (con lợn)

- Gồm 45 file âm thanh.
- Đặc điểm: Tiếng kêu của lợn là tiếng “ừ ừ”, có âm lượng trung bình và âm sắc trầm. Âm thanh thường ngắn, lặp lại nhiều lần và có sự dao động nhẹ về biên độ. Tiếng lợn mang tính đều đặn, không có sự thay đổi lớn về cao độ.

m, Wolf (con chó sói)

- Gồm 50 file âm thanh.
- Đặc điểm: Tiếng kêu của chó sói là tiếng tru, có âm lượng lớn và vang xa. Âm thanh cao, kéo dài và có sự biến đổi dần về cao độ trong quá trình phát âm. Tiếng tru thường ổn định nhưng mang tính lan tỏa, có thể kéo dài liên tục và lặp lại theo chu kỳ.

3.2. Tiền xử lý

Do dữ liệu âm thanh được thu thập từ nhiều nguồn khác nhau, các tệp đầu vào có sự không đồng nhất về tần số lấy mẫu, số kênh (mono/stereo) và định dạng mã hóa. Những khác biệt này có thể ảnh hưởng đáng kể đến quá trình xử lý tín hiệu và làm giảm độ chính xác của các đặc trưng trích rút. Vì vậy, bước tiền xử lý dữ liệu đóng vai trò quan trọng nhằm chuẩn hóa và làm sạch dữ liệu trước khi đưa vào các bước phân tích tiếp theo.

Trong hệ thống đề xuất, quá trình tiền xử lý được thực hiện theo chuỗi các bước tuần tự: chuẩn hóa dữ liệu → cắt bỏ khoảng lặng → phân đoạn tín hiệu → lọc theo năng lượng → gom cụm các đoạn tương tự. Quy trình này giúp đảm bảo tính nhất quán của dữ liệu, đồng thời giảm nhiễu và tối ưu hóa biểu diễn tín hiệu.

3.2.1. Chuẩn hóa dữ liệu

Ở giai đoạn này, tất cả các tệp âm thanh đầu vào được chuyển đổi về cùng một định dạng thống nhất nhằm đảm bảo tính đồng nhất của dữ liệu. Hệ thống thực hiện các thao tác sau:

```
TARGET_SR = 22050

def standardize_dataset():

    """Tạo folder DatasetCleaned"""

    if not os.path.exists(CLEANED_DATASET_PATH):

        os.makedirs(CLEANED_DATASET_PATH)

    valid_exts = ('.wav', '.mp3', '.flac')

    files = [f for f in os.listdir(RAW_DATASET_PATH) if
f.lower().endswith(valid_exts)]

    for filename in files:

        try:

            raw_path = os.path.join(RAW_DATASET_PATH, filename)

            # librosa sẽ xử lý các đuôi file khác nhau ở đây

            y, _ = librosa.load(raw_path, sr=TARGET_SR, mono=True)

            base_name = os.path.splitext(filename)[0]

            # Mọi định dạng input đều được lưu về .wav trong
            DatasetCleaned để trích xuất đặc trưng ổn định nhất

            out_path = os.path.join(CLEANED_DATASET_PATH,
f"{base_name}.wav")

            sf.write(out_path, y, TARGET_SR, subtype='PCM_16')

        except Exception as e:
```

```
print(f"Lỗi chuẩn hóa {filename}: {e}")
```

Cụ thể:

- Tất cả các tệp âm thanh được resample về cùng tần số lấy mẫu 22050 Hz, đảm bảo tính đồng nhất về miền thời gian.
- Tín hiệu được chuyển về dạng mono để giảm độ phức tạp và đảm bảo tính nhất quán.
- Các định dạng khác nhau (.mp3, .flac) được chuyển đổi về định dạng .wav để đảm bảo khả năng xử lý ổn định.
- Dữ liệu được lưu với chuẩn mã hóa PCM 16-bit nhằm đảm bảo chất lượng tín hiệu ổn định.

Quá trình này giúp loại bỏ sự khác biệt về định dạng và cấu trúc dữ liệu, từ đó tạo điều kiện thuận lợi cho các bước xử lý tiếp theo.

3.2.2. Xử lý tín hiệu âm thanh

Sau khi dữ liệu được chuẩn hóa về định dạng thống nhất, mỗi tín hiệu âm thanh tiếp tục trải qua các bước xử lý ở mức tín hiệu nhằm loại bỏ nhiễu, phân tích cấu trúc theo thời gian và chuẩn bị cho quá trình trích rút đặc trưng. Quy trình này bao gồm các bước: cắt bỏ khoảng lặng, phân đoạn tín hiệu theo cửa sổ trượt, lọc theo năng lượng và gom cụm các đoạn tương tự.

a, Cắt bỏ khoảng lặng

Tín hiệu âm thanh sau khi được tải vào hệ thống thường bao gồm cả các đoạn mang thông tin (tiếng kêu động vật) và các đoạn không mang ý nghĩa như khoảng lặng hoặc nhiễu nền. Để loại bỏ các thành phần không cần thiết ở biên của tín hiệu, hệ thống sử dụng hàm `librosa.effects.trim` nhằm cắt bỏ khoảng lặng dựa trên ngưỡng năng lượng.

Cụ thể, tín hiệu được đọc và chuẩn hóa về cùng tần số lấy mẫu và dạng mono, sau đó áp dụng hàm cắt khoảng lặng như sau:

```
TARGET_SR = 22050
```

```
TOP_DB_TRIM = 20
```

```
y, sr = librosa.load(file_path, sr=TARGET_SR, mono=True)
```

```
y_trimmed, _ = librosa.effects.trim(y, top_db=TOP_DB_TRIM)
```

Trong đó, tham số `top_db = 20` xác định ngưỡng năng lượng tương đối so với mức năng lượng cực đại của tín hiệu. Các đoạn có năng lượng thấp hơn ngưỡng này sẽ bị loại bỏ.

Tuy nhiên, hàm `librosa.effects.trim` ở đây chỉ loại bỏ các khoảng lặng ở đầu và cuối tín hiệu. Các đoạn khoảng lặng nằm bên trong tín hiệu không bị loại bỏ ở bước này mà sẽ được xử lý ở các bước tiếp theo thông qua cơ chế lọc theo năng lượng của từng frame.

Kết quả của bước này là tín hiệu được rút gọn, chỉ giữ lại phần trung tâm chứa nội dung chính. Việc loại bỏ khoảng lặng ở hai đầu giúp giảm độ dài tín hiệu cần xử lý, từ đó giảm chi phí tính toán. Đồng thời, quá trình này còn hạn chế ảnh hưởng của nhiễu nền và các đoạn không mang thông tin, góp phần nâng cao độ chính xác trong quá trình trích rút đặc trưng.

b, Phân đoạn tín hiệu

Sau khi loại bỏ khoảng lặng, tín hiệu được chia thành các đoạn ngắn có độ dài mỗi frame (window size) là 0.1 giây theo cửa sổ trượt với bước trượt (hop size) 0.05 giây nhằm phân tích đặc trưng theo thời gian.

```
WINDOW_DURATION = 0.1
```

```
HOP_DURATION = 0.05
```

```
window_size = int(WINDOW_DURATION * sr)
```

```
hop_size = int(HOP_DURATION * sr)
```

```
for i in range(0, len(y_trimmed) - window_size, hop_size):
```

```
    seg = y_trimmed[i: i + window_size]
```

Với tần số lấy mẫu $sr = 22050$, ta có:

- Số mẫu mỗi frame:

$$window_size = 0.1 \times 22050 = 2205 \text{ samples}$$

- Bước trượt:

$$hop_size = 0.05 \times 22050 = 1102 \text{ samples}$$

Cách chia này tạo ra các frame chồng lấn 50%, tức là mỗi frame kế tiếp sẽ chia sẻ một nửa dữ liệu với frame trước đó. Điều này mang lại hai lợi ích chính:

- Tăng độ mịn theo thời gian: giúp phát hiện tốt hơn các biến đổi nhỏ trong tín hiệu
- Giảm mất mát thông tin: hạn chế việc bỏ sót các đặc trưng nằm ở ranh giới giữa các frame

Mỗi frame được xem như một đơn vị phân tích độc lập, từ đó các đặc trưng sẽ được trích rút riêng biệt cho từng đoạn.

c, Lọc theo năng lượng

Không phải tất cả các frame đều chứa thông tin hữu ích. Một số frame vẫn có thể là nhiễu hoặc phần dư sau khi cắt khoảng lặng. Do đó, hệ thống tiếp tục áp dụng cơ chế lọc dựa trên năng lượng của từng frame.

```
all_rmse = librosa.feature.rms(y=y_trimmed)[0]
if len(all_rmse) == 0: return []
#Ngưỡng động
dynamic_threshold = max(0.0005, np.max(all_rmse) * 0.3)

rmse, zcr, centroid, bandwidth = compute_manual_features(seg, sr)

#Lọc bỏ đoạn có năng lượng thấp
if rmse < dynamic_threshold:
    if current_cluster is not None:
        segments.append(current_cluster)
        current_cluster = None
    continue
```

Năng lượng của frame được tính bằng chỉ số RMSE (Root Mean Square Energy). Sau đó, một ngưỡng động (dynamic threshold) được nhóm thiết lập như sau:

$$threshold = \max(0.0005, 0.3 \times RMSE_{max}) \quad (8)$$

Trong đó:

- $RMSE_{max}$ là giá trị năng lượng lớn nhất trong toàn bộ tín hiệu

- 0.3 là hệ số điều chỉnh thực nghiệm

Cách tiếp cận này cho phép ngưỡng thích nghi với từng tín hiệu cụ thể, thay vì sử dụng một giá trị cố định. Các frame có năng lượng nhỏ hơn ngưỡng sẽ bị loại bỏ, qua đó hệ thống chỉ giữ lại các đoạn âm thanh quan trọng, đồng thời loại bỏ những nhiễu yếu hoặc tín hiệu không đáng kể, góp phần giảm khối lượng dữ liệu cần xử lý.

d, Gom cụm các đoạn tương tự

Sau khi trải qua bước lọc theo năng lượng, các frame còn lại tiếp tục được gom nhóm thành các segment dựa trên mức độ tương đồng về đặc trưng. Mục tiêu của bước này là giảm số lượng đơn vị dữ liệu cần xử lý, đồng thời tạo ra các đoạn âm thanh có tính đại diện cao hơn cho từng vùng tín hiệu.

Cụ thể, với mỗi frame, hệ thống tiến hành trích rút một tập đặc trưng mô tả tín hiệu (các đặc trưng này sẽ được trình bày chi tiết trong Mục 3.3). Dựa trên các vector đặc trưng này, hệ thống so sánh frame hiện tại với cụm đang xét thông qua khoảng cách giữa hai vector.

Mỗi frame được biểu diễn bởi vector đặc trưng:

$$x = [rmse, zcr, mfcc, centroid, bandwidth, rolloff] \quad (9)$$

Hệ thống sử dụng khoảng cách Euclid chuẩn hóa giữa hai vector đặc trưng:

$$d = \frac{\|x_{new} - x_{cluster}\|}{\|x_{cluster}\| + \epsilon} \quad (11)$$

Trong đó:

x_{new} : vector của frame hiện tại

$x_{cluster}$: vector đại diện của cụm hiện tại

$\epsilon = 0.05$: hệ số tránh chia cho 0

Cơ chế gom cụm được thực hiện như sau:

- Nếu $d < 0.4$ (ngưỡng tương đồng): Frame được xem là tương tự, và được gộp vào cụm hiện tại bằng cách lấy trung bình:

$$x_{cluster} = \frac{x_{cluster} + x_{new}}{2} \quad (12)$$

- Nếu ≥ 0.4 : Kết thúc cụm hiện tại, tạo một cụm mới

Nếu mức độ sai khác giữa hai vector nhỏ hơn một ngưỡng xác định trước, frame mới sẽ được xem là tương đồng và được gộp vào cụm hiện tại bằng cách cập nhật lại giá trị đại diện (trung bình) của cụm. Ngược lại, nếu sai khác vượt quá ngưỡng, cụm hiện tại sẽ được kết thúc và một cụm mới sẽ được khởi tạo từ frame đang xét.

```
mfcc_val = np.mean(librosa.feature.mfcc(y=seg, sr=sr, n_mfcc=13))
rolloff = librosa.feature.spectral_rolloff(y=seg, sr=sr,
roll_percent=0.85) [0, 0]

new_feat = np.array([rmse, zcr, mfcc_val, centroid, bandwidth, rolloff],
dtype=np.float64)

if current_cluster is None:
    current_cluster = new_feat
else:
    dist = np.linalg.norm(new_feat - current_cluster)
    norm_dist = dist / (np.linalg.norm(current_cluster) + 0.05)

if norm_dist < SIMILARITY_THRESHOLD:
    current_cluster = (current_cluster + new_feat) / 2
else:
    segments.append(current_cluster)
    current_cluster = new_feat

#Thêm cụm cuối cùng
if current_cluster is not None:
    segments.append(current_cluster)
```

Quá trình này được thực hiện tuần tự theo thời gian, đảm bảo rằng mỗi segment thu được là một tập hợp các frame liên tiếp có đặc trưng tương đối đồng nhất. Nhờ đó, tín hiệu ban đầu được biểu diễn dưới dạng một số lượng nhỏ các segment mang tính khái quát cao hơn.

Việc gom cụm không chỉ giúp giảm kích thước dữ liệu mà còn làm mượt các đặc trưng theo thời gian, từ đó nâng cao tính ổn định và hiệu quả trong các bước xử lý và so

khớp âm thanh tiếp theo. Tuy nhiên, phương pháp này có thể bị ảnh hưởng bởi nhiễu cục bộ hoặc thay đổi đột ngột của tín hiệu.

3.3. Trích rút đặc trưng

Sau khi tín hiệu âm thanh được tiền xử lý và phân đoạn thành các segment, hệ thống tiến hành trích rút đặc trưng nhằm biểu diễn mỗi đoạn tín hiệu dưới dạng vector số. Các đặc trưng này được sử dụng làm cơ sở cho quá trình so sánh và nhận dạng âm thanh.

Trong hệ thống đề xuất, các đặc trưng được chia thành hai nhóm:

- Nhóm đặc trưng được tự cài đặt bằng NumPy
- Nhóm đặc trưng sử dụng thư viện Librosa

3.3.1. Trích rút đặc trưng thủ công bằng NumPy

Bốn đặc trưng bao gồm RMSE, ZCR, Spectral Centroid và Spectral Bandwidth được cài đặt trực tiếp bằng các phép toán số học cơ bản mà không sử dụng thư viện chuyên dụng. Việc này giúp kiểm soát tốt hơn quá trình tính toán và minh bạch hóa cách thức hoạt động của từng đặc trưng.

Cụ thể, hàm `compute_manual_features` được xây dựng như sau:

```
def compute_manual_features(seg, sr):  
    """Tính toán đặc trưng bằng NumPy"""  
  
    rmse = np.sqrt(np.mean(seg ** 2))  
  
    zcr = np.mean(np.abs(np.diff(np.sign(seg))) > 0)  
  
    magnitude = np.abs(np.fft.rfft(seg * np.hamming(len(seg))))  
    freqs = np.fft.rfftfreq(len(seg), 1 / sr)  
    mag_sum = np.sum(magnitude) + 1e-10  
  
    centroid = np.sum(freqs * magnitude) / mag_sum  
    bandwidth = np.sqrt(np.sum(((freqs - centroid) ** 2) * magnitude) /  
mag_sum)  
  
    return rmse, zcr, centroid, bandwidth
```

Trong đó:

- RMSE được tính trực tiếp từ giá trị trung bình bình phương của tín hiệu trong frame.
- ZCR được xác định thông qua việc đếm số lần thay đổi dấu giữa các mẫu liên tiếp bằng phép toán sai phân (np.diff) và hàm dấu (np.sign).
- Spectral Centroid và Spectral Bandwidth được tính toán dựa trên phổ biên độ thu được từ biến đổi Fourier nhanh (FFT). Trước khi thực hiện FFT, tín hiệu được nhân với cửa sổ Hamming nhằm giảm hiện tượng rò rỉ phổ.

Việc sử dụng NumPy cho phép tối ưu hóa tốc độ tính toán đồng thời giữ được tính chính xác của các đặc trưng cơ bản.

3.3.2. Trích rút đặc trưng bằng thư viện Librosa

Đối với các đặc trưng phức tạp hơn như MFCC và Spectral Rolloff, việc tự cài đặt từ đầu đòi hỏi nhiều bước xử lý tín hiệu nâng cao (biến đổi Mel, log, DCT, tích lũy năng lượng,...), do đó hệ thống sử dụng thư viện Librosa để đảm bảo tính chính xác và độ tin cậy.

Cách cài đặt trong hệ thống như sau:

```
mfcc_val = np.mean(librosa.feature.mfcc(y=seg, sr=sr, n_mfcc=13))
rolloff    = librosa.feature.spectral_rolloff(y=seg, sr=sr,
roll_percent=0.85)[0, 0]
```

Trong đó:

- MFCC: hệ thống sử dụng 13 hệ số MFCC và lấy giá trị trung bình để biểu diễn cho toàn bộ frame, giúp giảm chiều dữ liệu.
- Spectral Rolloff: được tính với ngưỡng 85% năng lượng phổ, phản ánh tần số mà tại đó phần lớn năng lượng tín hiệu được tích lũy.

Việc sử dụng Librosa giúp đơn giản hóa quá trình triển khai, đồng thời đảm bảo các đặc trưng được tính toán theo tiêu chuẩn phổ biến trong lĩnh vực xử lý âm thanh.

3.3.3. Tạo vector đặc trưng

Sau khi tính toán các đặc trưng, hệ thống kết hợp chúng thành một vector đặc trưng duy nhất cho mỗi frame:

```
mfcc_val = np.mean(librosa.feature.mfcc(y=seg, sr=sr, n_mfcc=13))  
rolloff      = librosa.feature.spectral_rolloff(y=seg, sr=sr,  
roll_percent=0.85)[0, 0]
```

Vector này bao gồm:

- 2 đặc trưng miền thời gian: RMSE, ZCR
- 4 đặc trưng miền tần số: Centroid, Bandwidth, Rolloff, MFCC

Cách biểu diễn này cho phép mô tả tín hiệu âm thanh một cách đầy đủ từ nhiều góc độ khác nhau, đồng thời đảm bảo tính gọn nhẹ để phục vụ cho các bước xử lý tiếp theo như gom cụm và so khớp.

CHƯƠNG 4. THIẾT KẾ CƠ SỞ DỮ LIỆU VÀ HỆ THỐNG TÌM KIẾM

4.1. Thiết kế CSDL đặc trưng

Trong hệ thống này, cơ sở dữ liệu được xây dựng nhằm lưu trữ các đặc trưng âm thanh đã được trích xuất từ tập dữ liệu tiếng động vật. Hệ quản trị được sử dụng là SQLite do tính đơn giản, nhẹ và phù hợp với quy mô bài toán. Dữ liệu được tổ chức dưới dạng một bảng duy nhất mang tên Segments, trong đó mỗi dòng tương ứng với một đoạn âm thanh nhỏ được tách ra từ file gốc.

Mỗi segment được biểu diễn bởi một vector đặc trưng bao gồm các thuộc tính như năng lượng tín hiệu (RMSE), tần suất đổi dấu (ZCR), hệ số MFCC trung bình, trọng tâm phổ (spectral centroid), độ rộng phổ (bandwidth) và tần số roll-off. Việc lưu trữ theo dạng phân đoạn thay vì toàn bộ file giúp hệ thống tăng độ chính xác trong quá trình so sánh, do mỗi file âm thanh thường có nhiều biến đổi theo thời gian.

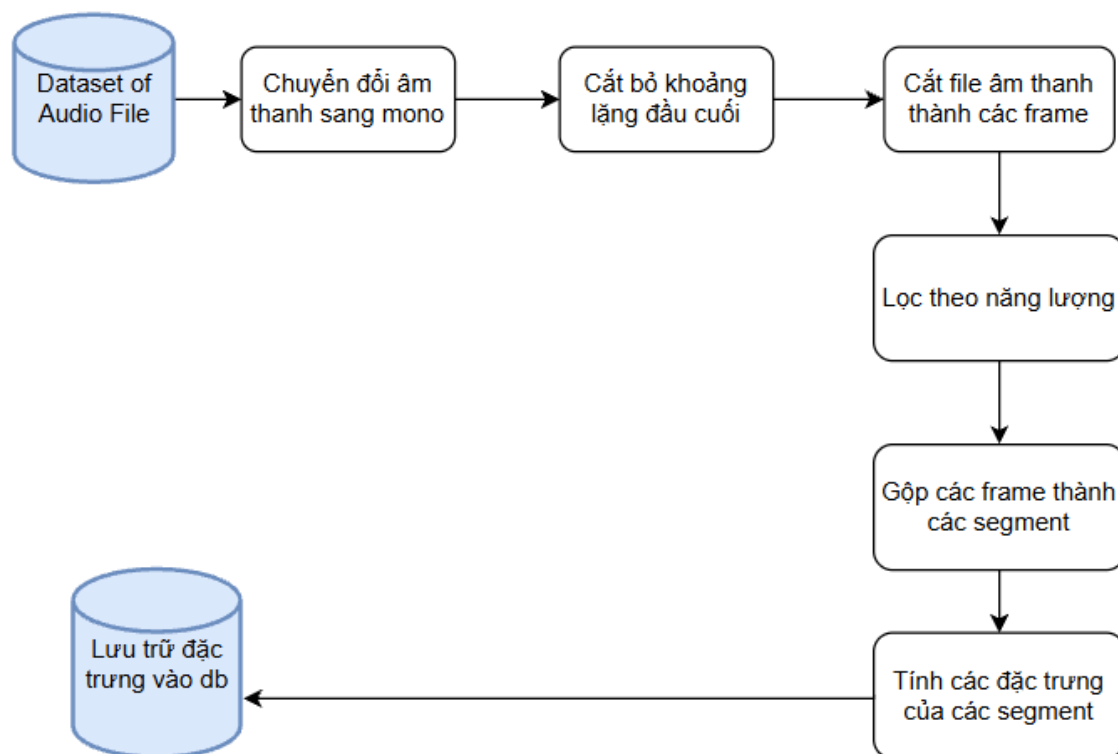
file_name	rmse	zcr	mfcc	centroid	bandwidth	rolloff
Filter	Filter	Filter	Filter	Filter	Filter	Filter
Bear_01.wav	0.372078009677352	0.0346741719600726	7.32881692098454	966.488629222348	1596.50969727477	1911.05074882507
Bear_01.wav	0.191381314536557	0.0305800533121597	2.79029804468155	1170.85250686918	1817.21752699091	2813.44757080078
Bear_01.wav	0.169019485425679	0.0540966788567129	-4.25718519301881	1305.03300343829	1714.10919001576	2897.66645953059
Bear_02.wav	0.159158647060394	0.0744101633393829	-1.74783480167389	706.812329190432	1457.89545025413	5534.033203125
Bear_02.wav	0.299112260341644	0.0206442831215971	7.26558482646942	546.993915765466	1191.6616011027	2589.36767578125
Bear_02.wav	0.382459282975061	0.0181488203266788	9.75835609436035	508.471086758724	1006.69593036088	753.662109375
Bear_02.wav	0.386979967355728	0.0181488203266788	11.4738712310791	519.529341956494	1175.08337493186	3951.3427734375
Bear_02.wav	0.331277623772621	0.0183756805807622	10.4002251625061	576.947436634155	1231.64430253255	764.4287109375
Bear_02.wav	0.157208830118179	0.0285843920145191	6.43040704727173	1132.60801437736	1836.03929556893	2282.51953125
Bear_02.wav	0.203158117830753	0.0319872958257713	3.25775969028473	861.304662693046	1439.69965941982	2196.38671875
Bear_02.wav	0.341993570327759	0.0322141560798548	5.86467313766479	778.899738214445	1161.97832676155	1108.9599609375
Bear_02.wav	0.435179352760315	0.0254083484573503	7.86256790161133	650.521255320662	1067.93591357623	2433.251953125
Bear_02.wav	0.413631170988083	0.0226860254083485	9.93990993499756	554.607091626701	1031.96704787027	1195.0927734375
Bear_02.wav	0.239219792187214	0.0395871143375681	5.80020713806152	1093.917004442	1690.38860686629	2831.6162109375
Bear_03.wav	0.348782867867612	0.0663412651435866	6.0646460941648	1391.15864352884	1823.66725044715	2775.85946330801
Bear_04.wav	0.331693384796381	0.0274500907441016	11.6572749614716	1569.2279981254	2344.28614215774	4753.45458984375
Bear_04.wav	0.426509737968445	0.0381125226860254	15.023099899292	1399.48882571083	2160.56605015664	1916.455078125
Bear_04.wav	0.378469518764646	0.0403759139851953	9.78466872498205	1329.42190685965	2080.68321911571	3005.01850412402
Bear_04.wav	0.325060784816742	0.0421960072595281	9.35336589813232	1184.31179827084	1877.17646568958	4909.5703125
Bear_04.wav	0.36929777264595	0.0394736842105263	9.58149671554565	1276.77601723842	1970.68472991582	2858.53271484375
Bear_04.wav	0.356818437576294	0.0381125226860254	9.57144737243652	1139.87877616995	1870.23421351412	4435.83984375
Bear_04.wav	0.418602675199509	0.0263157894736842	11.1250600814819	1339.70049898121	2045.61355067761	2454.78515625
Bear_04.wav	0.344197218888439	0.0373610480943739	9.98344172537327	1407.62050603541	2091.84439881687	3255.88760375977

Hình 4.1. CSDL đặc trưng

Các đặc trưng được lựa chọn đều có ý nghĩa trong việc mô tả tín hiệu âm thanh. RMSE phản ánh cường độ âm thanh, ZCR giúp phân biệt các tín hiệu có tính chất dao động nhanh, MFCC đóng vai trò quan trọng trong nhận dạng âm thanh, trong khi các đặc trưng phổ như centroid, bandwidth và rolloff giúp mô tả cấu trúc tần số của tín hiệu.

Sự kết hợp này tạo nên một bộ đặc trưng cân bằng giữa độ chính xác và chi phí tính toán.

4.2. Sơ đồ khối hệ thống



Hình 4.2. Sơ đồ khối hệ thống

Hệ thống tìm kiếm được xây dựng theo một chuỗi các bước xử lý tuần tự, bắt đầu từ dữ liệu đầu vào là file âm thanh do người dùng cung cấp. Đầu tiên, tín hiệu được chuẩn hóa về cùng tần số lấy mẫu nhằm đảm bảo tính đồng nhất. Sau đó, hệ thống tiến hành loại bỏ các đoạn im lặng ở đầu và cuối để giữ lại phần tín hiệu có thông tin.

Tiếp theo, tín hiệu được chia thành các đoạn nhỏ với độ dài cố định. Trên mỗi đoạn, hệ thống trích xuất các đặc trưng âm thanh và tạo thành các vector đại diện. Các vector này sẽ được so sánh với dữ liệu đã lưu trong cơ sở dữ liệu. Kết quả so sánh được tổng hợp, sắp xếp và cuối cùng trả về danh sách các file âm thanh có độ tương đồng cao nhất với đầu vào.

Toàn bộ quá trình này tạo thành một pipeline xử lý rõ ràng, trong đó mỗi bước đều có vai trò cụ thể nhằm đảm bảo kết quả cuối cùng có ý nghĩa và độ chính xác tương đối cao.

4.3. Chuẩn hóa đặc trưng

Trong quá trình trích rút đặc trưng, các thuộc tính thu được có miền giá trị và đơn vị đo khác nhau. Ví dụ, RMSE thường có giá trị nhỏ trong khoảng $[0,1]$, trong khi các đặc trưng phổ như spectral centroid hoặc bandwidth có thể đạt tới hàng nghìn Hz. Sự khác biệt về thang đo này có thể gây mất cân bằng trong quá trình tính toán độ tương đồng, khi một số đặc trưng có giá trị lớn sẽ chi phối kết quả so sánh.

Để khắc phục vấn đề này, hệ thống áp dụng phương pháp chuẩn hóa Min–Max nhằm đưa toàn bộ các đặc trưng về cùng một miền giá trị $[0,1]$. Công thức chuẩn hóa được sử dụng như sau:

$$x' = \frac{x - x_{min}}{x_{max} - x_{min}} \quad (13)$$

Trong đó:

- x là giá trị đặc trưng ban đầu
- x_{min}, x_{max} là giới hạn dưới và trên của đặc trưng
- x' là giá trị sau chuẩn hóa

Trong hệ thống, các giới hạn này được xác định dựa trên phạm vi giá trị vật lý của từng đặc trưng. Cụ thể:

- RMSE: $[0,0.5]$
- ZCR: $[0,1]$
- MFCC: $[-100,100]$
- Centroid, Bandwidth, Rolloff: $[0,11025]$

Việc chuẩn hóa được thực hiện ngay sau khi vector đặc trưng được tạo ra. Mỗi vector sau khi chuẩn hóa sẽ được biểu diễn lại dưới dạng:

$$x = [rmse', zcr', mfcc', centroid', bandwidth', rolloff'] \quad (14)$$

Sau khi thực hiện bước chuẩn hóa, các vector đặc trưng không được ghi đè lên dữ liệu ban đầu mà được lưu trữ trong một cơ sở dữ liệu riêng biệt. Cách tiếp cận này nhằm đảm bảo tính toàn vẹn của dữ liệu gốc, đồng thời tạo điều kiện thuận lợi cho việc so sánh, đánh giá và mở rộng hệ thống trong các giai đoạn tiếp theo.

Cụ thể, hệ thống triển khai một quy trình di cư dữ liệu (data migration) từ cơ sở dữ liệu chứa các đặc trưng thô sang một cơ sở dữ liệu mới, trong đó toàn bộ các vector đặc trưng đã được chuẩn hóa theo phương pháp Min–Max. Cấu trúc của bảng dữ liệu trong cơ sở dữ liệu mới được giữ nguyên như bảng ban đầu, tuy nhiên các giá trị lưu trữ đã được chuyển đổi về cùng miền giá trị $[0,1]$, như minh họa sau:

file_name	rmse	zcr	mfcc	centroid	bandwidth	rolloff
Filter	Filter	Filter	Filter	Filter	Filter	Filter
Bear_01.wav	0.744156019354705	0.0346741719600726	0.536644084604923	0.0876633677299181	0.144808135807235	0.173337936401367
Bear_01.wav	0.382762629073113	0.0305800533121597	0.513951490223408	0.106199773865686	0.164826986575139	0.25518798828125
Bear_01.wav	0.338038970851358	0.0540966788567129	0.478714074034906	0.118370340447917	0.155474756464014	0.262826889753342
Bear_02.wav	0.318317294120789	0.0744101633393829	0.491260825991631	0.0641099618313317	0.132235414989037	0.501953125
Bear_02.wav	0.598224520683289	0.0206442831215971	0.536327924132347	0.0496139606136477	0.108087220054666	0.23486328125
Bear_02.wav	0.764918565750122	0.0181488203266788	0.548791780471802	0.0461198264633763	0.0913102884681067	0.068359375
Bear_02.wav	0.773959934711456	0.0181488203266788	0.557369356155396	0.0471228428078452	0.106583526070917	0.3583984375
Bear_02.wav	0.662555247545242	0.0183756805807622	0.55200112581253	0.0523308332547986	0.111713768937193	0.0693359375
Bear_02.wav	0.314417660236359	0.0285843920145191	0.532152035236359	0.10273088565781	0.166534176468837	0.20703125
Bear_02.wav	0.406316235661507	0.0319872958257713	0.516288798451424	0.078122871899596	0.130585003121979	0.19921875
Bear_02.wav	0.683987140655518	0.0322141560798548	0.529323365688324	0.0706485023323759	0.105394859570209	0.1005859375
Bear_02.wav	0.87035870552063	0.0254083484573503	0.539312839508057	0.0590041954939376	0.0968649354717667	0.220703125
Bear_02.wav	0.827262341976166	0.0226860254083485	0.549699549674988	0.0503044981067302	0.0936024533215662	0.1083984375
Bear_02.wav	0.478439584374428	0.0395871143375681	0.529001035690308	0.0992214970015418	0.153323229647736	0.2568359375
Bear_03.wav	0.697565735735225	0.0663412651435866	0.530323230470824	0.126182189889238	0.16541199550541	0.25177863612771
Bear_04.wav	0.663386769592762	0.0274500907441016	0.558286374807358	0.142333605272146	0.212633663687777	0.43115234375
Bear_04.wav	0.85301947593689	0.0381125226860254	0.57511549949646	0.126937761969236	0.195969709764775	0.173828125
Bear_04.wav	0.756939037529293	0.0403759139851953	0.54892334362491	0.120582485882962	0.188724101507094	0.27256403665524
Bear_04.wav	0.650121569633484	0.0421960072595281	0.546766829490662	0.107420571271731	0.170265439064814	0.4453125
Bear_04.wav	0.738595545291901	0.0394736842105263	0.547907483577728	0.115807348502351	0.178746914278078	0.25927734375
Bear_04.wav	0.713636875152588	0.0381125226860254	0.547857236862183	0.103390365185483	0.169635756327811	0.40234375
Bear_04.wav	0.837205350399017	0.0263157894736842	0.55562530040741	0.121514784488091	0.185543179199783	0.22265625
Bear_04.wav	0.688394437776878	0.0373610480943739	0.549917208626866	0.127675329345615	0.189736453407426	0.295318603515625
Bear_04.wav	0.355793075128148	0.0812878888877896	0.55372438416489	0.212005850193357	0.224034512534431	0.439900464627892

Hình 4.3. CSDL đặc trưng sau chuẩn hóa

Quy trình di cư và chuẩn hóa dữ liệu được thực hiện tuần tự qua các bước sau:

- Truy xuất toàn bộ các vector đặc trưng từ cơ sở dữ liệu ban đầu
- Áp dụng hàm chuẩn hóa Min–Max cho từng vector đặc trưng
- Ghi lại các vector đã chuẩn hóa vào bảng SegmentsNormalized trong cơ sở dữ liệu mới

Việc tách biệt cơ sở dữ liệu chứa đặc trưng chuẩn hóa mang lại nhiều lợi ích quan trọng. Thứ nhất, dữ liệu gốc được bảo toàn hoàn toàn, giúp hỗ trợ quá trình kiểm tra, đối chiếu và tái sử dụng khi cần thiết. Thứ hai, dữ liệu trong cơ sở dữ liệu mới đã sẵn sàng cho các phép đo độ tương đồng, từ đó góp phần cải thiện hiệu năng truy vấn trong giai đoạn tìm kiếm. Cuối cùng, việc áp dụng cùng một quy tắc chuẩn hóa cho cả dữ liệu lưu trữ và dữ liệu đầu vào đảm bảo tính nhất quán trong toàn bộ hệ thống, qua đó nâng cao độ tin cậy và độ chính xác của kết quả so khớp. Sau khi các vector đặc trưng được chuẩn

hóa và lưu trữ, hệ thống có thể sử dụng trực tiếp các phép đo khoảng cách (như khoảng cách Euclid) để đánh giá mức độ tương đồng giữa các đoạn âm thanh.

4.4. Phương pháp đo độ tương đồng

Sau khi trích rút đặc trưng và chuẩn hóa, mỗi đoạn âm thanh được biểu diễn dưới dạng một vector đặc trưng trong không gian nhiều chiều. Để xác định mức độ giống nhau giữa các đoạn âm thanh, hệ thống sử dụng độ đo khoảng cách giữa các vector đặc trưng.

Trong bài tập lớn này, phương pháp được sử dụng là khoảng cách Euclid có chuẩn hóa. Khác với khoảng cách Euclid truyền thống, hệ thống tiến hành chuẩn hóa sai khác giữa các thành phần của vector nhằm giảm ảnh hưởng của sự khác biệt về thang đo giữa các đặc trưng.

Cụ thể, với hai vector đặc trưng:

$x = [x_1, x_2, \dots, x_d]$: vector của đoạn âm thanh đầu vào

$y = [y_1, y_2, \dots, y_d]$: vector của đoạn âm thanh trong cơ sở dữ liệu

Trước tiên, hệ thống vector sai khác đã được chuẩn hóa:

$$diff = \left[\frac{x_1 - y_1}{y_1 + \epsilon}, \frac{x_2 - y_2}{y_2 + \epsilon}, \dots, \frac{x_d - y_d}{y_d + \epsilon} \right] \quad (15)$$

Trong đó ϵ là một hằng số rất nhỏ nhằm tránh hiện tượng chia cho 0

Sau đó, khoảng cách giữa hai vector được tính bằng chuẩn Euclid:

$$d(x, y) = \sqrt{\sum_{i=1}^d \left(\frac{x_i - y_i}{y_i + \epsilon} \right)^2} \quad (16)$$

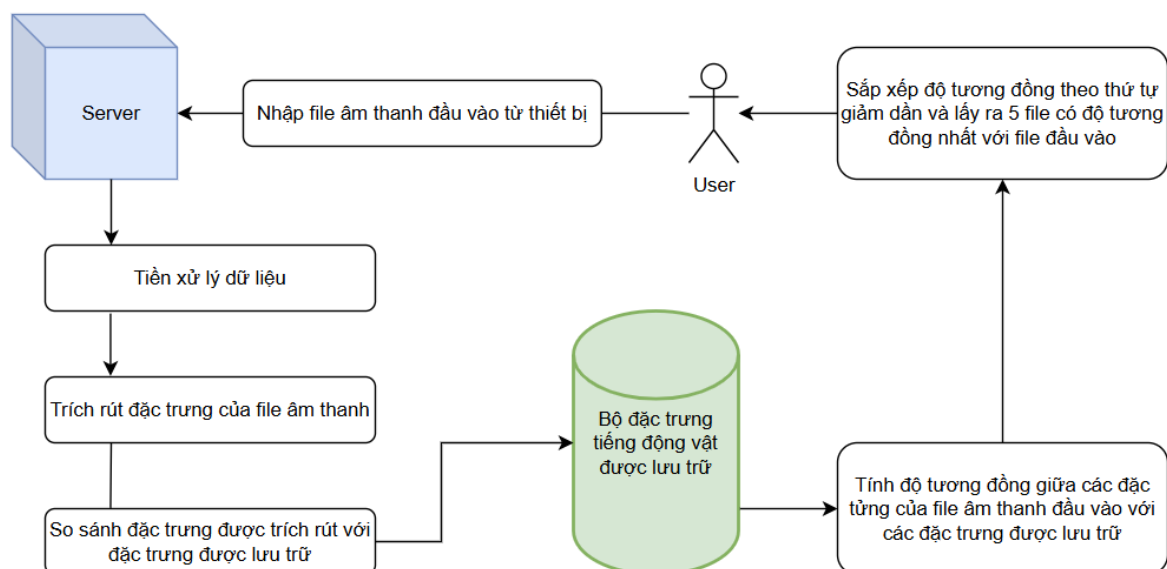
Khoảng cách này phản ánh mức độ sai khác giữa hai đoạn âm thanh, trong đó giá trị khoảng cách càng nhỏ thì mức độ tương đồng càng cao; ngược lại, khoảng cách lớn cho thấy hai đoạn âm thanh có sự khác biệt rõ rệt về đặc trưng. Trong quá trình tìm kiếm, mỗi segment của file đầu vào được so sánh với tất cả các segment trong cơ sở dữ liệu. Đối với mỗi file trong cơ sở dữ liệu, hệ thống chỉ giữ lại giá trị khoảng cách nhỏ nhất, xem như đại diện cho mức độ giống nhau giữa hai file. Cuối cùng, các kết quả

được sắp xếp theo thứ tự tăng dần của khoảng cách, từ đó đưa ra danh sách các tệp âm thanh có mức độ tương đồng cao nhất với tín hiệu đầu vào.

Phương pháp này tuy đơn giản nhưng hiệu quả trong các bài toán có quy mô vừa phải, đồng thời dễ triển khai và không yêu cầu các mô hình học máy phức tạp.

4.5. Quy trình tìm kiếm

Quy trình tìm kiếm bắt đầu khi người dùng tải lên một file âm thanh thông qua giao diện web. File này được hệ thống đọc vào, sau đó trải qua các bước tiền xử lý bao gồm chuẩn hóa tần số lấy mẫu và loại bỏ khoảng lặng. Tiếp theo, tín hiệu được chia thành các đoạn nhỏ và mỗi đoạn được trích xuất đặc trưng và chuẩn hóa tương tự như dữ liệu đã lưu trong cơ sở dữ liệu.



Hình 4.4. Quy trình tìm kiếm

Sau khi có các vector đặc trưng của file đầu vào, hệ thống tiến hành so sánh với toàn bộ dữ liệu hiện có. Với mỗi vector đầu vào, khoảng cách tới các vector trong cơ sở dữ liệu được tính toán. Kết quả được tổng hợp lại theo từng file và giá trị nhỏ nhất được sử dụng để đại diện cho mức độ tương đồng.

Cuối cùng, các file được sắp xếp theo thứ tự tăng dần của khoảng cách và hệ thống trả về năm kết quả có độ tương đồng cao nhất. Người dùng có thể xem tên file, giá trị khoảng cách và nghe trực tiếp các file này trên giao diện web.

4.6. Kết quả trung gian

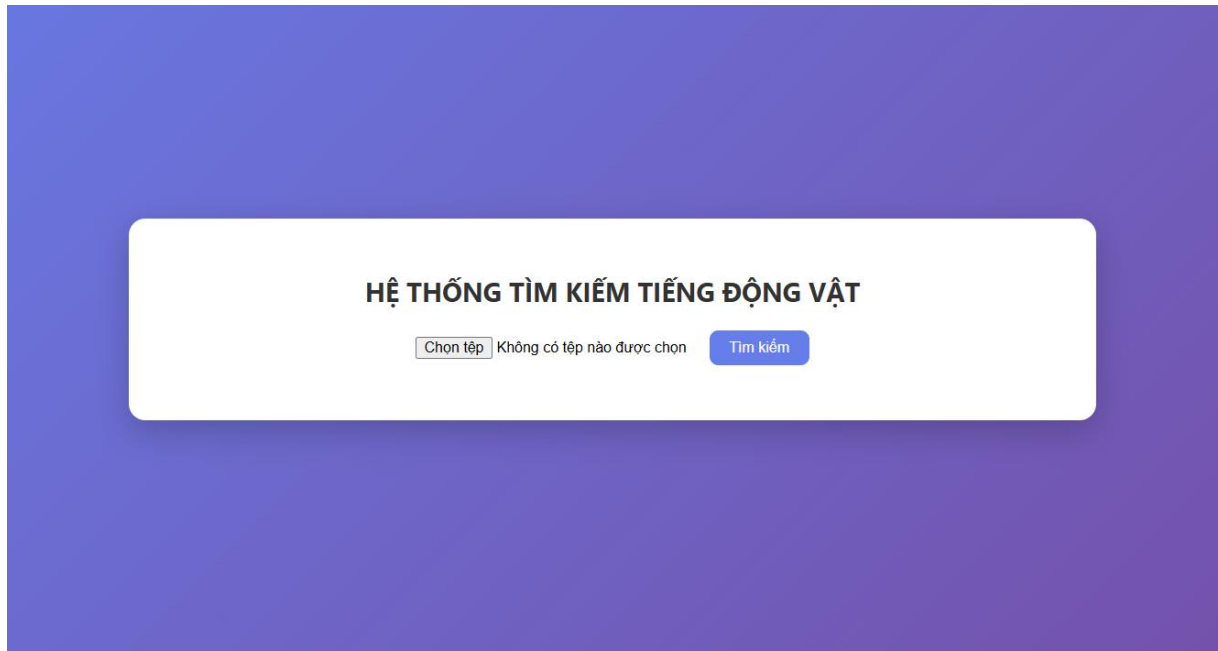
Trong quá trình thực nghiệm, khi sử dụng một file đầu vào cụ thể, hệ thống trả về danh sách các file có độ tương đồng cao nhất kèm theo giá trị khoảng cách tương ứng. Quan sát cho thấy các file thuộc cùng loại âm thanh thường có khoảng cách nhỏ hơn so với các file thuộc loại khác.

Kết quả này cho thấy các đặc trưng đã lựa chọn có khả năng phản ánh tương đối chính xác bản chất của tín hiệu âm thanh. Tuy nhiên, vẫn tồn tại một số trường hợp mà các âm thanh khác nhau nhưng có đặc điểm phổ tương tự dẫn đến kết quả chưa hoàn toàn chính xác. Điều này là do hạn chế của phương pháp đặc trưng và cách đo khoảng cách hiện tại.

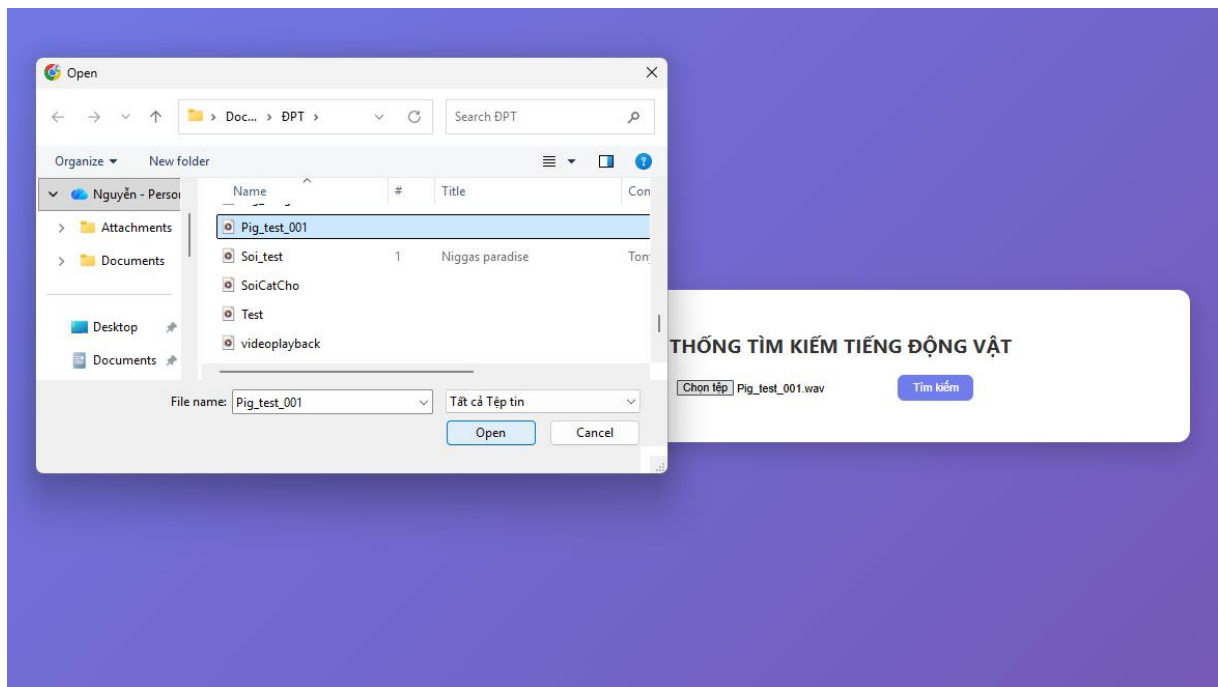
CHƯƠNG 5. DEMO VÀ ĐÁNH GIÁ

5.5. Demo hệ thống

Hệ thống được triển khai dưới dạng một ứng dụng web chạy trên môi trường cục bộ. Người dùng có thể truy cập giao diện thông qua trình duyệt và thực hiện thao tác tải lên file âm thanh từ máy tính cá nhân. Sau khi nhấn nút tìm kiếm, hệ thống sẽ xử lý dữ liệu và hiển thị kết quả trong thời gian ngắn.

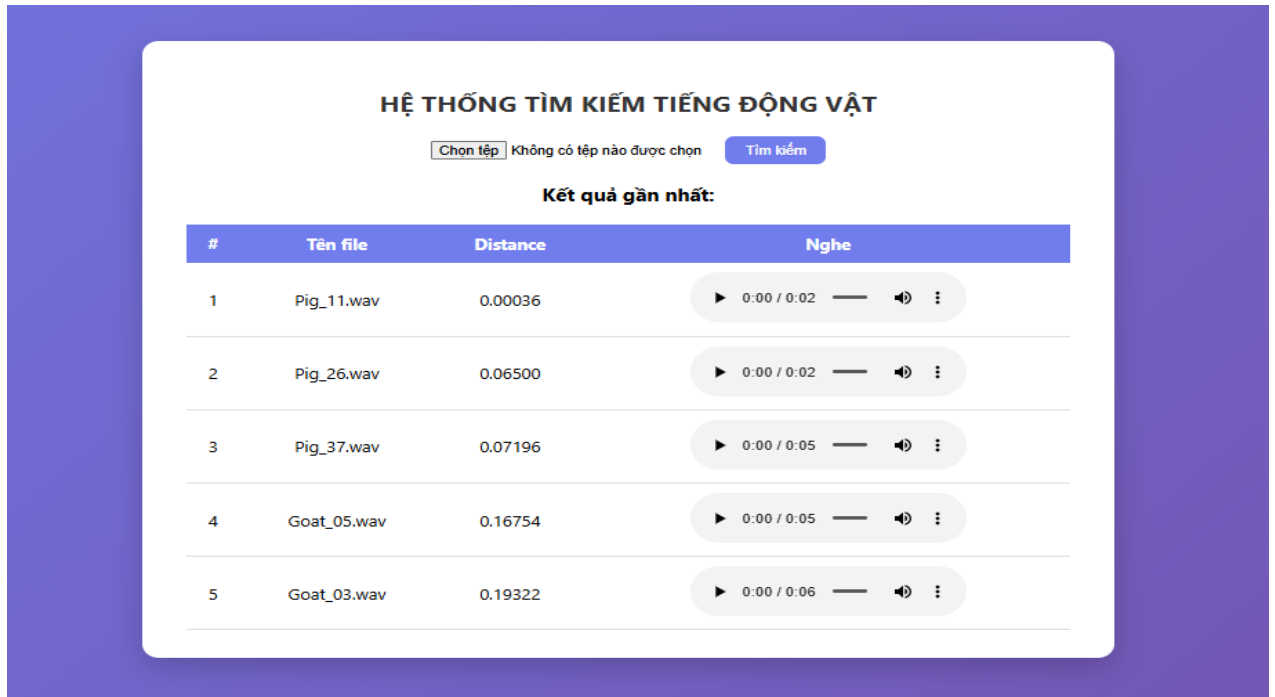


Hình 5.1. Giao diện chính hiển thị hệ thống tìm kiếm tiếng động vật



Hình 5.2. Giao diện hệ thống khi chọn file tìm kiếm từ thiết bị cục bộ

Giao diện hiển thị danh sách năm file âm thanh có độ tương đồng cao nhất với file đầu vào. Mỗi kết quả bao gồm tên file, giá trị khoảng cách và một trình phát âm thanh cho phép người dùng nghe trực tiếp. Việc tích hợp chức năng phát âm thanh giúp quá trình đánh giá kết quả trở nên trực quan hơn.



Hình 5.3. Giao diện hệ thống khi ấn nút “Tìm kiếm”

Quá trình demo cho thấy hệ thống hoạt động ổn định, phản hồi nhanh và dễ sử dụng, phù hợp với mục tiêu minh họa của bài toán.

Ngoài giao diện web hiển thị kết quả, hệ thống còn ghi nhận toàn bộ quá trình xử lý và truy cập thông qua cửa sổ Terminal. Cụ thể, mỗi khi người dùng thực hiện thao tác tải file và tìm kiếm, các yêu cầu HTTP tương ứng sẽ được hiển thị dưới dạng log. Các dòng log này bao gồm địa chỉ truy cập (127.0.0.1), thời gian thực hiện, phương thức yêu cầu (GET, POST) và đường dẫn tài nguyên được truy cập.

```
(.venv) PS C:\Users\ADM\Documents\PythonProject> python .\app.py
127.0.0.1 - - [04/May/2026 20:27:02] "GET /audio/Monkey_02.wav HTTP/1.1" 206 -
127.0.0.1 - - [04/May/2026 20:27:03] "GET /audio/Monkey_02.wav HTTP/1.1" 206 -
127.0.0.1 - - [04/May/2026 20:27:03] "GET /audio/Monkey_02.wav HTTP/1.1" 206 -
127.0.0.1 - - [04/May/2026 20:27:03] "GET /audio/Monkey_02.wav HTTP/1.1" 206 -
127.0.0.1 - - [04/May/2026 20:27:03] "GET /audio/Monkey_02.wav HTTP/1.1" 206 -
127.0.0.1 - - [04/May/2026 20:27:24] "POST / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2026 20:27:24] "GET /audio/Pig_11.wav HTTP/1.1" 206 -
127.0.0.1 - - [04/May/2026 20:27:24] "GET /audio/Pig_26.wav HTTP/1.1" 206 -
127.0.0.1 - - [04/May/2026 20:27:24] "GET /audio/Pig_37.wav HTTP/1.1" 206 -
127.0.0.1 - - [04/May/2026 20:27:24] "GET /audio/Goat_05.wav HTTP/1.1" 206 -
127.0.0.1 - - [04/May/2026 20:27:24] "GET /audio/Goat_03.wav HTTP/1.1" 206 -
127.0.0.1 - - [04/May/2026 20:35:59] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2026 20:41:30] "POST / HTTP/1.1" 200 -
127.0.0.1 - - [04/May/2026 20:41:30] "GET /audio/Pig_11.wav HTTP/1.1" 206 -
127.0.0.1 - - [04/May/2026 20:41:30] "GET /audio/Pig_26.wav HTTP/1.1" 206 -
127.0.0.1 - - [04/May/2026 20:41:30] "GET /audio/Pig_37.wav HTTP/1.1" 206 -
127.0.0.1 - - [04/May/2026 20:41:30] "GET /audio/Goat_05.wav HTTP/1.1" 206 -
127.0.0.1 - - [04/May/2026 20:41:30] "GET /audio/Goat_03.wav HTTP/1.1" 206 -
```

Hình 5.4. Log hệ thống khi chạy ứng dụng web (app.py) trên localhost

Đặc biệt, khi người dùng phát các file âm thanh kết quả trên giao diện web, hệ thống sẽ gửi các yêu cầu GET tới đường dẫn chứa file âm thanh, và các yêu cầu này cũng được ghi lại trong Terminal. Điều này cho phép theo dõi chính xác những file nào đang được truy xuất và kiểm tra trạng thái phản hồi của hệ thống (ví dụ mã trạng thái 200 thể hiện yêu cầu thành công).

Việc hiển thị log trong Terminal giúp hỗ trợ quá trình debug và đánh giá hoạt động của hệ thống một cách trực quan. Thông qua đó, người phát triển có thể phát hiện lỗi, kiểm tra luồng xử lý và đảm bảo hệ thống hoạt động ổn định trong quá trình demo.

5.6. Đánh giá

Hệ thống có ưu điểm là thiết kế đơn giản, dễ triển khai và không yêu cầu tài nguyên tính toán lớn. Việc sử dụng các đặc trưng cơ bản kết hợp với phương pháp đo khoảng cách trực tiếp giúp giảm độ phức tạp, đồng thời vẫn đạt được kết quả tương đối tốt trong việc tìm kiếm âm thanh tương tự.

Tuy nhiên, phương pháp hiện tại vẫn tồn tại một số hạn chế. Việc so sánh theo kiểu duyệt toàn bộ cơ sở dữ liệu khiến thời gian xử lý có thể tăng đáng kể khi quy mô dữ liệu lớn. Ngoài ra, việc sử dụng giá trị trung bình của MFCC làm mất đi nhiều thông tin quan trọng của tín hiệu, dẫn đến độ chính xác chưa cao trong một số trường hợp. Hệ thống cũng chưa xử lý tốt các tín hiệu bị nhiễu hoặc có sự biến đổi lớn.

Nhìn chung, hệ thống đáp ứng tốt yêu cầu của bài toán ở mức cơ bản, nhưng vẫn còn nhiều tiềm năng để cải thiện.

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

Hệ thống đã thực hiện thành công việc xây dựng cơ sở dữ liệu đặc trưng cho âm thanh động vật, đồng thời triển khai được chức năng tìm kiếm các file âm thanh tương tự dựa trên nội dung. Kết quả đạt được cho thấy phương pháp tiếp cận dựa trên trích xuất đặc trưng và so sánh khoảng cách là khả thi và phù hợp với bài toán.

Trong tương lai, hệ thống có thể được cải tiến theo nhiều hướng khác nhau nhằm nâng cao hiệu quả. Một trong những hướng quan trọng là sử dụng các phương pháp đo tương đồng tiên tiến hơn như cosine similarity hoặc Dynamic Time Warping để cải thiện độ chính xác. Bên cạnh đó, việc sử dụng đầy đủ vector MFCC thay vì giá trị trung bình sẽ giúp giữ lại nhiều thông tin hơn từ tín hiệu.

Ngoài ra, có thể áp dụng các kỹ thuật tối ưu tìm kiếm như sử dụng cấu trúc dữ liệu không gian hoặc các thư viện chuyên dụng để giảm thời gian truy vấn. Một hướng phát triển nâng cao hơn là áp dụng các mô hình học sâu để học biểu diễn đặc trưng âm thanh, từ đó cải thiện đáng kể khả năng nhận diện và tìm kiếm.

Tổng thể, hệ thống hiện tại là một nền tảng tốt để tiếp tục phát triển thành một ứng dụng hoàn chỉnh với độ chính xác và hiệu năng cao hơn trong tương lai.